

Design Report: Safe Semantic Planner in a Finite Cartesian State Space

Course: PCCST503 - Machine Learning

Department: Computer Science and Engineering

1. Problem Formulation & Architecture

The objective of this project is to construct a safe semantic path planner operating within a finite Cartesian state space $S = \{s_1, s_2, \dots, s_n\} \subset \mathbb{R}^d$. The planner computes an optimal, collision-free path from an initial state s_I to a goal state s_G while completely avoiding bad states $B = \{b_1, b_2, \dots, b_k\}$ and balancing multi-objective metrics.

Class Specifications

The implementation adheres to object-oriented principles:

- **State:** Encapsulates a unique integer ID and a spatial coordinate vector embedding $\mathbf{s}_i \in \mathbb{R}^d$.
- **Transition:** Defines directed edges (s_i, s_j) characterized by cost, safety factor, reliability rating, and an availability flag.
- **PlanningProblem & PlanningResult:** Structurally define input state spaces and output trajectories, execution metrics, and min-safety distances.

2. Theoretical Design & Formulas

2.1 Admissible Heuristic Function

To guarantee optimal pathfinding via A^* / D^* search principles, the heuristic function $h(u, v)$ measures the straight-line Euclidean distance between states u and v in $\mathbb{R}^d$ space:

$$h(u, v) = \|\mathbf{x}_u - \mathbf{x}_v\|_2 = \sqrt{\sum_{k=1}^d (x_{u,k} - x_{v,k})^2}$$

Because Euclidean distance represents the absolute shortest spatial path between points, $h(u, v)$ is admissible ($h(u, v) \leq h^*(u, v)$) and consistent, ensuring search optimality.

2.2 Safety Computation & Multi-Objective Weighting

Safety is evaluated dynamically by calculating the minimum Euclidean clearance from a target state v to all bad states B :

$$\text{DistToBad}(v) = \min_{b \in B} \|\mathbf{x}_v - \mathbf{x}_b\|_2$$

The effective edge cost $w(e)$ for transition $e = (u, v)$ fuses transition cost, safety clearance, and reliability:

$$w(e) = \alpha \cdot \text{Cost}(e) + \beta \cdot \left(\frac{1}{\text{DistToBad}(v)} + \epsilon \right) + \gamma \cdot (1 - \text{Reliability}(e))$$

where α, β, γ are tuning hyper-parameters and $\epsilon = 10^{-5}$ prevents division by zero.

3. Complexity Analysis

- Time Complexity:** $O((|V| + |E|) \log |V|)$
Where $|V| = |S|$ is the state space size and $|E| = |T|$ is the number of available transitions. Priority queue operations (min-heap push/pop) take $O(\log |V|)$, and each edge is visited at most once during graph search.
- Space Complexity:** $O(|V| + |E|)$

Required to store spatial lookup maps (dict), adjacency list graph structures, and parent pointers for path reconstruction.

4. Dynamic Replanning Strategy

In real-world dynamic environments, goal locations, state safety statuses, and transition availabilities change periodically.

- Graph Invalidation:** When edge availability or bad state sets update, the planner invalidates affected adjacency lists.
- Incremental Adaptation:** Instead of recalculating unchanged spatial heuristics, the planner updates local cost structures (g -values) and re-executes search from modified sub-graphs, ensuring minimal replanning overhead.

5. Experimental Evaluation

Below are the logged metrics generated by running `main.py` across all mandatory test scenarios:

Test Case	Scenario	Path Returned	Total Cost	Min Safety Dist	Nodes Explored	Execution Time	Goal Success
TC1	Basic Reachability	[0, 1, 2, 3]	3.0000	∞	4	0.0667 ms	100%
TC2	Bad State Avoidance	[0, 3, 4, 5]	11.2716	1.4142	5	0.0892 ms	100%

TC3	Safety Margin	[0, 3, 2]	9.1666	2.0000	3	0.0390 ms	100%
TC4	Dynamic Transition	[0, 1, 3, 2]	3.0000	\$\infty\$	4	0.0353 ms	100%
TC5	Goal Update	[0, 1, 3]	2.0000	\$\infty\$	3	0.0313 ms	100%
TC6	Transition Addition	[0, 2]	0.5000	\$\infty\$	2	0.0387 ms	100%